

introJulia

August 27, 2026

Introduction à Julia

Fabian Bastin (fabian.bastin@umontreal.ca), 2017-2026

0.1 Premiers pas en Julia

La page officielle du langage de programmation Julia est <http://julialang.org>

- Conçu pour répondre au problème de double langage (un pour le développement rapide, un pour la production), en alliant efficacité avec simplicité de code.
- Concerne avant tout le calcul scientifique.
- JIT (just-in-time): le code Julia est compilé au moment de son exécution, un peu à l'image de Java.
- Version actuelle: 1.12.x
- Croissance exponentielle des librairies disponibles.
- Peut s'interfacer avec des langages tels que Python, C++, Fortran, R,...
- Versions commerciales pour l'infonuagique: JuliaHub

0.1.1 Pourquoi pas Python?

- Python est idéal pour le prototypage, mais souffre de son côté interprété pour les boucles.
- Julia est un langage fortement typé, permettant des performances proches de C/C++ (à condition de veiller à la qualité de son implémentation).

0.2 Installation

L'interpréteur et compilateur de Julia peut être téléchargé à l'adresse <http://julialang.org/downloads/>

Julia peut être installé sur les OS majeurs: Windows, MacOS X, Linux. Sous Windows, il est possible de lancer Julia via le menu démarrer ou, si présente, en cliquant sur l'icône adéquate. Sous Linux, pour lancer l'interpréteur Julia en mode interactif, il suffit d'entrer au niveau du terminal

```
julia
```

Il est également possible d'exécuter un code julia en indiquant le nom du fichier, par exemple

```
julia hello.jl
```

où le fichier "hello.jl" contient la commande

```
[ ]: println("Hello World!")
```

0.3 Tester Julia

Il est également possible de programmer en Julia directement en ligne, notamment à l'adresse <https://repl.it> ou [cocalc](https://cocalc.com/) (<https://cocalc.com/>).

0.4 Environnements de développement

L'environnement de développement le plus utilisé et recommandé est **Visual Studio Code** avec l'extension Julia : - Visual Studio Code: <https://code.visualstudio.com/>

(Note : Les anciens environnements Juno et Atom ne sont plus maintenus et ont été remplacés par VS Code).

0.5 Librairies

Un nombre important de librairies officielles sont disponibles, mais ne sont pas installées par défaut avec Julia. Il existe plusieurs méthodes pour utiliser le gestionnaire de librairies. Nous allons travailler ici avec les commandes du gestionnaire Pkg, qui nous devons au préalable importer avec la commande

```
[ ]: import Pkg
```

Nous pouvons à présent ajouter une librairie avec la commande `Pkg.add("Nom de la librairie")`. Par exemple, pour pouvoir réaliser des graphes des fonctions qui nous intéressent, nous utiliserons la librairie Plots. Pour l'ajouter, entrons

Astuce : En pratique, les développeurs Julia utilisent très souvent le **mode Pkg du REPL**. Dans le terminal Julia, appuyez sur la touche `]` pour y accéder (l'invite devient `pkg>`), puis tapez simplement `add Plots`.

```
[ ]: Pkg.add("Plots")
```

0.5.1 Où se trouvent localement les librairies?

Sous Linux et MacOS, les sources des librairies installées sont disponibles par défaut dans le sous-répertoire `$HOME/.julia`

Sous Windows, il suffit de remplacer `$HOME` par le répertoire utilisateur.

La liste des librairies officielles ("registered packages") est disponible à l'adresse <https://julialang.org/packages/>.

Il est conseillé de mettre à jour régulièrement les librairies installées avec la commande

```
[ ]: Pkg.update()
```

Il est également possible d'importer une librairie non officielle disponible sur GitHub en remplaçant le nom de la librairie par son URL, par exemple

```
[ ]: Pkg.add(url="https://github.com/JLChartrand/RandomDataStreams.jl")
```

0.6 Bloc-notes

Julia s'intègre aussi à l'environnement de bloc-notes, permettant de mélanger texte et code, tout en offrant un environnement interactif. Un fichier de bloc-notes se reconnaît à l'extension de fichier `ipynb`.

Outre Jupyter, **Pluto.jl** (<https://plutojl.org/>) est un environnement de notebook réactif très populaire et natif à Julia, particulièrement apprécié pour l'enseignement et le prototypage.

Pour utiliser Julia avec les bloc-notes Jupyter classiques, il est nécessaire d'ajouter la librairie IJulia.

```
[ ]: Pkg.add("IJulia")
```

Pour lire, écrire et modifier des bloc-notes, il est possible d'utiliser **Jupyter** ou **Nteract**. Le premier s'exécute à l'intérieur du navigateur internet tandis que le second est une application indépendante. Visual Studio supporte également les bloc-notes dans une certaine étendue.

0.6.1 Jupyter

L'environnement Jupyter (<http://www.jupyter.org>) est une application web permettant de créer des documents combinant texte, codes, équations, et graphiques. Il est également possible d'exporter le code source ainsi que de créer des documents pdf et même des présentations interactives.

Le nom vient d'une origine double. Il évoque la planète Jupiter ainsi que les langages majeurs supportés par Jupyter: Julia, Python et R. Sa documentation peut être consultée à l'adresse <https://jupyter.org/documentation>.

L'installation directe de Jupyter est dépendante de l'OS, mais peut être également réalisée à travers la suite Anaconda, téléchargeable à l'adresse: <https://anaconda.com>, laquelle offre un environnement intégré pour Python.

Une fois la librairie installée, nous la chargerons en mémoire et utiliserons la fonction 'notebook' pour démarrer Jupyter.

```
[ ]: using IJulia
      notebook()
```

Si aucune version de Jupyter n'est détectée, Julia installera une version minimale de Jupyter. Pour démarrer Jupyter dans le navigateur internet, depuis la ligne de commande, entrez

```
jupyter notebook
```

Si la version exécutée par défaut de Jupyter ne propose pas de noyau Julia, ou si celle-ci n'est pas à jour, il est possible de l'ajouter via le terminal Julia avec la commande

```
[ ]: IJulia.installkernel("Julia")
```

Si la commande `notebook()` échoue à lancer automatiquement le navigateur internet, dans le fichier `.jupyter/jupyter_notebook_config.py`, vérifiez la présence de la ligne suivante (non commentée)

```
c.NotebookApp.use_redirect_file = False
```

0.7 Nteract

Une alternative à Jupyter est **Nteract**, disponible à l'adresse <https://nteract.io/> Son avantage est de fonctionner en dehors du navigateur internet. Par contre, son support de formatage de texte est moins avancé.

Nteract étant une interface, le support des bloc-notes doit être installé au sein du langage de programmation utilisé.

Comme indiqué à la page <https://nteract.io/kernels/julia>, il est nécessaire d'installer le noyau Julia pour pouvoir exécuter du code.

```
[ ]: # From a Julia prompt
      using Pkg
      Pkg.update("IJulia")
      IJulia.installkernel("Julia nteract")
```

0.8 MyBinder

Si le notebook Jupyter est hébergé sur une plateforme de développement coopératif telle que <https://github.com>, il est possible de l'exécuter en renseignant son URL sur le site <https://mybinder.org/>. Il convient cependant au préalable d'ajouter dans le fichier `Project.toml`, qui doit être placé à la racine du projet, la version minimale de Julia dans la section `[compat]`, par exemple, en exigeant une installation minimale de la version 1.10.0 de Julia:

```
[compat]
```

```
julia = "1.10"
```

À titre d'exemple, cette introduction ainsi que des notebook d'introduction à la programmation mathématique peuvent être consultés via le lien <https://mybinder.org/v2/gh/fbastin/optim/HEAD>

0.9 Visual Studio

Il est également possible d'utiliser Visual Studio pour développer en Julia, comme expliqué sur la page officielle <https://code.visualstudio.com/docs/languages/julia>

De plus, Visual Studio supporte les bloc-notes jupyter: <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

0.10 Boîtes à outil spécifiques

Diverses orientations spécifiques existent, permettant de spécialiser l'usage de Julia pour des besoins spécifiques. Il serait difficile d'être exhaustif, et nous ne citerons ici qu'un cas particulier, mais il existe actuellement des ensembles d'outils pour les principaux sujets de recherche scientifique actifs.

0.10.1 Optimisation

Julia supporte de nombreux algorithmes d'optimisation et propose un langage de modélisation algébrique. Les détails peuvent être trouvés à l'adresse <http://www.juliaopt.org>

En outre, le langage JuMP offre de modéliser les programmes mathématiques de manière intuitive.

0.10.2 GPU

Julia offre un support des GPU grâce aux librairies reprises à dans le projet JuliaGPU: <https://juliagpu.org/>

0.10.3 Exporter vers d'autres formats

Il est également possible d'exporter les bloc-notes, directement depuis l'interface de Jupyter, ou en ligne de commandes. Par exemples, pour créer des slides, on pourra utiliser

```
jupyter nbconvert your_slides.ipynb --to slides
```

Pour appeler directement le navigateur internet, entrez

```
jupyter nbconvert your_slides.ipynb --to slides --post serve
```

Il est également possible d'exporter vers des formats tels que PDF, HTML, etc.

Le principe général reste d'appeler jupyter avec la commande nbconvert, et d'indiquer le format de conversion, par exemple

```
jupyter nbconvert your_slides.ipynb --to PDF
```

0.11 Tutoriaux

De nombreux tutoriaux existent, mais il faut veiller à consulter un tutorial à jour pour la dernière version de Julia, comme le langage est encore en cours d'évolution. Le plus simple est de consulter la page <https://julialang.org/learning/>

Dans l'interprétation, il est possible d'accéder à de la documentation en ligne en introduisant le caractère '?' dans la ligne de commande.

Il est possible de suivre des cours d'introduction sur <https://juliaacademy.com/> En particulier, un cours d'introduction est disponible à l'adresse <https://juliaacademy.com/p/intro-to-julia>

1 Support

Il existe différents canaux de communication avec la communauté Julia. Mentionnons ici <https://discourse.julialang.org/>

2 Débuter avec Julia

Des bloc-notes interactifs sont disponibles sur Julia Academy (<https://juliaacademy.com/>), en particulier <https://github.com/JuliaAcademy/Introduction-to-Julia>.

2.0.1 Affectation de variables

La syntaxe de base consiste à écrire le nom de variable suivi du signe d'égalité et de la valeur à affecter à la variable, par exemple

```
[ ]: the_answer_to_life_the_universe_and_everything = 42
```

Note: Google est d'accord https://www.google.ca/search?sxsrf=ALeKk00YlrkNlc4nJGBesWkETj6KN4G3yw%3Aab&ved=0ahUKEwilt_rr6LHrAhVqTd8KHTBOAREQ4dUDCAw&uact=5

Julia est fortement typé, pourtant nous n'avons pas précisé le type! Par défaut, Julia infère le type de la valeur assignée, et nous pouvons le connaître comme suit:

```
[ ]: typeof(the_answer_to_life_the_universe_and_everything)
```

Simplifions le nom de la variable, en la transformant au passage en réel:

```
[ ]: life = Float64(the_answer_to_life_the_universe_and_everything)
```

```
[ ]: typeof(life)
```

Jupyter (et Julia) supporte le code $\LaTeX$ et même les émoticônes! Pour le code $\LaTeX$, il suffit d'écrire la commande $\LaTeX$ puis d'appuyer sur tabulation. Les émoticônes s'obtiennent en écrivant en entourant le nom de l'émoticône de $\backslash$: et :, par exemple $\backslash$:alien:, suivi de tabulation. Si le nom n'est pas connu en entier, après $\backslash$: et les premières lettres de l'émoticône entrée, l'appui sur tabulation permet d'afficher une liste déroulante où vous pouvez choisir l'émoticône voulue.

```
[ ]: = 1.0
      = "alien"
println(+length( ))
```

```
[ ]: typeof( )
```

```
[ ]: = 1.0
      = -1.0
      = 0.0
      + ==
```

```
[ ]: !=
```

2.1 Types

Les types sont organisés hiérarchiquement, avec une structure arborescente. Il est possible de connaître le type dont dépend directement un type avec la commande `supertype`:

```
[ ]: println(supertype{Int64}, " ", supertype{Float64})
```

À la racine se trouve le type `Any`, dont dépendent directement un grand nombre de types, qui peuvent être obtenus avec la commande `subtypes`:

```
[ ]: subtypes{Any}
```

Nous pouvons exposer l'arborescence à l'aide de la fonction récursive suivante, que nous appliquons au type `Number`, qui a comme prédécesseur direct le type `Any`.

```
[ ]: # Tiré de https://en.wikibooks.org/wiki/Introducing_Julia/Types
function showtypetree(T, level=0)
    println("\t" ^ level, T)
```

```

    for t in subtypes(T)
        showtypetree(t, level+1)
    end
end

showtypetree(Number)

```

```
[ ]: typeof(5/9)
```

3 Tableaux

Les variables peuvent être groupées en tableaux, à une ou plusieurs dimensions. Il est à noter que les indices commencent à 1 en Julia.

```
[ ]: x = [i for i = 1:10]
```

```
[ ]: x[1]
```

3.1 Structures de contrôle

Comme la plupart des langages, Julia propose des structures conditionnelles et des boucles. Notez l'utilisation du mot-clé `end` pour fermer les blocs, et l'absence de deux-points `:` ou d'accolades `{}`.

```
[ ]: x = 5
if x < 0
    println("Négatif")
elseif x == 0
    println("Zéro")
else
    println("Positif")
end

```

```
[ ]: for i in 1:3
    println("Itération ", i)
end

```

4 Fonctions

La manière générique d'écrire une fonction en Julia est d'utiliser le mot-clé `function` suivi de la liste des paramètres à lui passer, pour terminer sur le mot-clé `end`.

```
[ ]: function hello()
    println("Bonjour !")
end

```

Remarquons qu'en Julia, même `function` est en fait un type de variable:

```
[ ]: typeof(hello)
```

Ainsi, une fonction peut elle-même renvoyer comme résultat une fonction:

```
[ ]: function compound_rate(rate::Float64)
    r = rate
    function compound(duration::Float64)
        return (1+r)^duration-1
    end
    return compound
end
```

```
[ ]: c = compound_rate(0.05)
      c(3.0)
```

Pour les fonctions n'ayant qu'une ligne d'instruction, il est ainsi possible de les déclarer comme une affectation de variable.

```
[ ]: hello(name) = println("Bonjour $(name)!")
```

```
[ ]: hello("user")
```

Cela permet en particulier d'écrire certaines expressions mathématiques de manière plus commode, p.e.

```
[ ]: f2(x) = x^2
```

Nous aurions aussi pu déclarer de telles fonctions comme “anonymes”.

```
[ ]: f_2 = x -> x^2
```

```
[ ]: f_2(3)
```

4.1 Type des arguments

Si le type des arguments n'est pas précisé, Julia essaye d'ajuster automatiquement le comportement de la fonction, pour autant que les expressions aient du sens.

```
[ ]: A = rand(3, 3)
      A
```

```
[ ]: f2(A)-A*A
```

```
[ ]: f2("tou")
```

4.1.1 Dispatch Multiple

Le véritable cœur de Julia est le **dispatch multiple**. Au lieu d'avoir une seule fonction abstraite qui gère tous les cas avec des `if`, vous pouvez définir *plusieurs méthodes* pour une même fonction, spécialisées par type. Julia choisira automatiquement la méthode la plus optimisée lors de l'exécution.


```
[ ]: saluer(x::Int) = println("C'est un entier : ", x)
      saluer(x::String) = println("C'est du texte : ", x)
```

```
[ ]: saluer(42)
```

```
[ ]: saluer("Bonjour")
```

4.2 Broadcasting (Vectorisation)

L'une des syntaxes les plus puissantes et courantes en Julia est le **broadcasting**, signalé par un point `..`. Il permet d'appliquer n'importe quelle fonction ou opérateur élément par élément sur une structure de données (comme un tableau), évitant ainsi d'écrire des boucles explicites.

```
[ ]: valeurs = [1, 2, 3]
      # Ajoute 10 à chaque élément (remarquez le point avant le +)
      valeurs .+ 10
```

```
[ ]: # Applique la fonction sinus à chaque élément
      sin.(valeurs)
```

4.3 Macros et Performance

Vous rencontrerez souvent des symboles précédés d'un `@` en Julia, comme `@time` ou `@show`. Ce sont des **macros**, un mécanisme puissant permettant de générer du code avant l'exécution.

Puisque Julia est taillé pour la performance, la macro `@time` est particulièrement utile pour évaluer le temps d'exécution et l'allocation mémoire de vos instructions.

```
[ ]: # Générer un grand tableau aléatoire et chronométrer son tri
      grand_tableau = rand(10^6);
      @time sort(grand_tableau);
```

4.4 Compilation statique

La librairie [PackageCompiler](#) offre de compiler le code Julia de manière statique voire, avec certains ajustements, de créer une librairie C, comme expliqué dans la [documentation](#) du paquet.

```
[ ]:
```